Chủ đề 7: Kiến trúc ứng dụng Android

Nội dung chính:

7.1. Nguyên tắc phân tách nhiệm vụ trong kiến trúc ứng dụng.

7.2. Triển khai mô hình ViewModel và LiveData.

7.3. Luồng dữ liệu một chiều (UDF) và Single Source of Truth.

Mục tiêu cần đạt được:

Hiểu nguyên tắc thiết kế kiến trúc ứng dụng Android theo mô hình hiện đại.

Biết cách sử dụng ViewModel và LiveData để quản lý dữ liệu và trạng thái UI.

Vận dụng UDF và các công cụ hỗ trợ như Dependency Injection để xây dựng ứng dụng dễ bảo trì và kiểm thử.

7.1. Kiến trúc ứng dụng

7.1.1. Kiến trúc ứng dụng Android

7.1.1.1. Trải nghiệm của người dùng trên ứng dụng điện thoại di động

Một ứng dụng Android thông thường có nhiều thành phần, bao gồm Activity, Fragment, Service, Content Provider và Broadcast Reciever. Những thành phần này được mô tả trong file manifest của ứng dụng. hệ điều hành Android sau đó sử dụng file này để quyết định làm thế nào tích hợp ưng dụng vào trải nghiệm người dùng tổng thể của thiết bị. Vì một ứng dụng Android thông thường có thể chứa nhiều thành phần và người dùng thường tương tác với nhiều ứng dụng trong thời gian ngắn, nên các ứng dụng cần thích ứng với các loại quy trình làm việc và tác vụ khác nhau do người dùng điều khiển.

Hãy nhớ rằng các thiết bị di động cũng bị hạn chế về tài nguyên, vì vậy bất cứ lúc nào, hệ điều hành cũng có thể tắt một số tiến trình ứng dụng để nhường chỗ cho tiến trình mới.

Với các điều kiện của môi trường này, các thành phần ứng dụng có thể được khởi chạy riêng lẻ và không theo thứ tự, và hệ điều hành hoặc người dùng có thể hủy chúng bất cứ lúc nào. Vì những sự kiện này không nằm trong tầm kiểm soát của người lập trình, do đó không nên lưu trữ hoặc giữ trong bộ nhớ bất kỳ dữ liệu hoặc trạng thái ứng dụng nào trong các thành phần ứng dụng của mình và các thành phần ứng dụng không nên phụ thuộc vào nhau.

7.1.1.2. Nguyên tắc kiến trúc chung

Không nên sử dụng các thành phần ứng dụng để lưu trữ dữ liệu và trạng thái ứng dụng. Khi các ứng dụng Android ngày càng lớn, phải xác định một kiến trúc cho phép

ứng dụng có thể mở rộng quy mô, tăng tính mạnh mẽ của ứng dụng và giúp ứng dụng dễ kiểm tra hơn.

Kiến trúc ứng dụng xác định ranh giới giữa các phần của ứng dụng và trách nhiệm mà mỗi phần phải có. Để đáp ứng các nhu cầu nêu trên, nên thiết kế kiến trúc ứng dụng của mình theo một số nguyên tắc cụ thể.

Phân tách các nhiệm vụ

Nguyên tắc quan trọng nhất cần tuân theo là tách biệt các nhiệm vụ. Không nên viết toàn bộ mã trong một Activity hoặc một Fragment, các lớp dựa trên UI này chỉ nên chứa logic xử lý các tương tác của UI và hệ điều hành. Bằng cách giữ cho các lớp này càng tinh gọn càng tốt, chúng ta có thể tránh được nhiều vấn đề liên quan đến vòng đời thành phần và cải thiện khả năng kiểm thử của các lớp này.

Hãy nhớ rằng không hoàn toàn kiểm soát được các lớp triển khai của Activity và Fragment; Đây là những thành phần được quản lý bởi hệ điều hành Android, và hệ điều hành có thể tạo ra hoặc hủy chúng mà ta không thể kiểm soát hoàn toàn, dựa trên tình trạng hệ thống hoặc cách người dùng tương tác với ứng dụng.

Nói cách khác, dù chúng ta có thể triển khai logic trong các lớp này, nhưng quyền kiểm soát thực sự đối với vòng đời và sự tồn tại của chúng nằm trong tay hệ điều hành. Vì vậy, không nên xây dựng toàn bộ ứng dụng phụ thuộc quá nhiều vào chúng, để tránh gặp phải các vấn đề khó quản lý hoặc lỗi không lường trước khi hệ điều hành hủy hoặc tạo lại các thành phần này.

7.1.1.3. Điều khiển giao diện người dùng dựa trên mô hình dữ liệu

Một nguyên tắc quan trọng khác là nên Điều khiển giao diện người dùng dựa trên mô hình dữ liệu, tốt nhất là các mô hình dữ liệu lưu trữ lâu dài. Các mô hình dữ liệu đại diện cho dữ liệu của một ứng dụng. Chúng độc lập với các thành phần giao diện người dùng và các thành phần khác trong ứng dụng . Điều này có nghĩa là chúng không bị ràng buộc với vòng đời của các thành phần UI và ứng dụng, nhưng vẫn sẽ bị hủy khi hệ điều hành quyết định loại bỏ tiến trình của ứng dụng ra khỏi bộ nhớ.

Các mô hình dữ liệu lưu trữ lâu dài là lý tưởng vì những lý do sau:

Người dùng sẽ không mất dữ liệu nếu hệ điều hành Android hủy ứng dụng để giải phóng tài nguyên.

Ứng dụng vẫn có thể hoạt động trong trường hợp kết nối mạng không ổn định hoặc không khả dụng.

Nếu chúng ta xây dựng kiến trúc ứng dụng dựa trên các lớp mô hình dữ liệu, ứng dụng sẽ dễ kiểm tra và vững chắc hơn.

7.1.1.4. Nguồn thông tin duy nhất (Single source of truth)

Khi một loại dữ liệu mới được định nghĩa trong ứng dụng , nên chỉ định một "Nguồn thông tin duy nhất" (Single Source of Truth - SSOT) cho loại dữ liệu đó. SSOT là thành phần sở hữu dữ liệu đó và chỉ có SSOT mới có thể sửa đổi hoặc thay đổi nó. Để đạt được điều này, SSOT cung cấp dữ liệu thông qua một kiểu dữ liệu bất biến và để sửa đổi dữ liệu, SSOT cung cấp các hàm hoặc nhận các sự kiện mà các loại khác có thể gọi đến.

Mô hình này mang lại nhiều lợi ích:

Nó tập trung tất cả các thay đổi của một loại dữ liệu cụ thể vào một nơi.

Nó bảo vệ dữ liệu để các loại khác không thể can thiệp vào.

Nó giúp cho các thay đổi dữ liệu trở nên dễ theo dõi hơn, do đó lỗi sẽ dễ dàng phát hiện hơn.

Trong một ứng dụng ưu tiên chế độ ngoại tuyến, nguồn thông tin cho dữ liệu ứng dụng thường là một cơ sở dữ liệu. Trong một số trường hợp khác, nguồn thông tin có thể là một ViewModel hoặc thậm chí là giao diện người dùng (UI).

7.1.1.5. Luồng dữ liệu một chiều (Unidirectional Data Flow)

Trong UDF (Unidirectional Data Flow), trạng thái của ứng dụng chỉ di chuyển theo một hướng, trong khi các sự kiện sửa đổi dữ liệu sẽ chảy theo hướng ngược lại. Điều này giúp duy trì tính nhất quán và dễ dàng theo dõi luồng dữ liệu trong ứng dụng.

Trong Android, dữ liệu và trạng thái thường di chuyển từ các thành phần có phạm vi rộng hơn (chẳng hạn như ViewModel hoặc Repository) đến các thành phần có phạm vi hẹp hơn (chẳng hạn như Fragment hoặc Activity). Ngược lại, các sự kiện do người dùng kích hoạt (như nhấn nút, nhập văn bản) thường bắt đầu từ giao diện người dùng và truyền đến Single Source of Truth (SSOT) – nơi dữ liệu được xử lý, cập nhật và phát lại theo hướng bất biến.

Ví dụ, khi ứng dụng tải dữ liệu từ một API hoặc cơ sở dữ liệu, dữ liệu này sẽ chảy từ Repository → ViewModel → UI (Activity/Fragment/Compose). Khi người dùng thực hiện một hành động như nhấn nút "Lưu", sự kiện này sẽ chảy từ UI → ViewModel → Repository, nơi dữ liệu được cập nhật trước khi phản hồi lại UI.

Mô hình UDF giúp đảm bảo rằng dữ liệu luôn nhất quán, giảm thiểu lỗi do cập nhật trạng thái không đồng bộ, dễ dàng gỡ lỗi hơn và mang lại lợi ích từ mô hình SSOT, nơi mọi thay đổi dữ liệu đều diễn ra ở một nơi duy nhất.

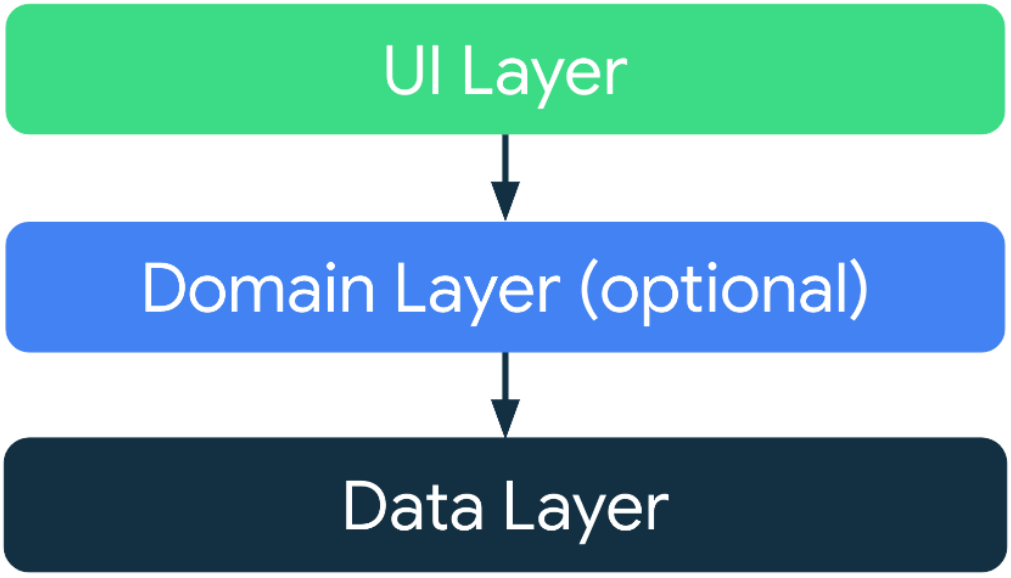
7.1.1.5. Kiến trúc ứng dụng được khuyến nghị

Dựa trên các nguyên tắc kiến trúc phổ biến đã được đề cập trong phần trước, mỗi ứng dụng nên có ít nhất hai lớp:

Lớp giao diện người dùng (UI) hiển thị dữ liệu ứng dụng trên màn hình.

Lớp dữ liệu chứa logic nghiệp vụ của ứng dụng và cung cấp dữ liệu ứng dụng.

có thể thêm một lớp bổ sung gọi là lớp miền (domain layer) để đơn giản hóa và tái sử dụng các tương tác giữa lớp UI và lớp dữ liệu.



Hình 7.1: Kiến trúc ứng dụng

7.1.1.6. Kiến trúc ứng dụng hiện đại

Kiến trúc ứng dụng hiện đại này khuyến khích sử dụng các kỹ thuật sau, cùng với những kỹ thuật khác:

Kiến trúc phản ứng và phân lớp.

Luồng dữ liệu một chiều (Unidirectional Data Flow - UDF) trong tất cả các lớp của ứng dụng.

Lớp giao diện người dùng (UI) với các thành phần quản lý trạng thái để kiểm soát sự phức tạp của UI.

Coroutines và flows.

Các thực tiễn tốt nhất về tiêm phụ thuộc (Dependency Injection).

Để biết thêm thông tin, hãy xem các phần sau đây, các trang Kiến trúc khác trong mục lục, và trang khuyến nghị chứa tóm tắt về các thực tiễn tốt nhất quan trọng nhất.

Lớp giao diện người dùng (UI)

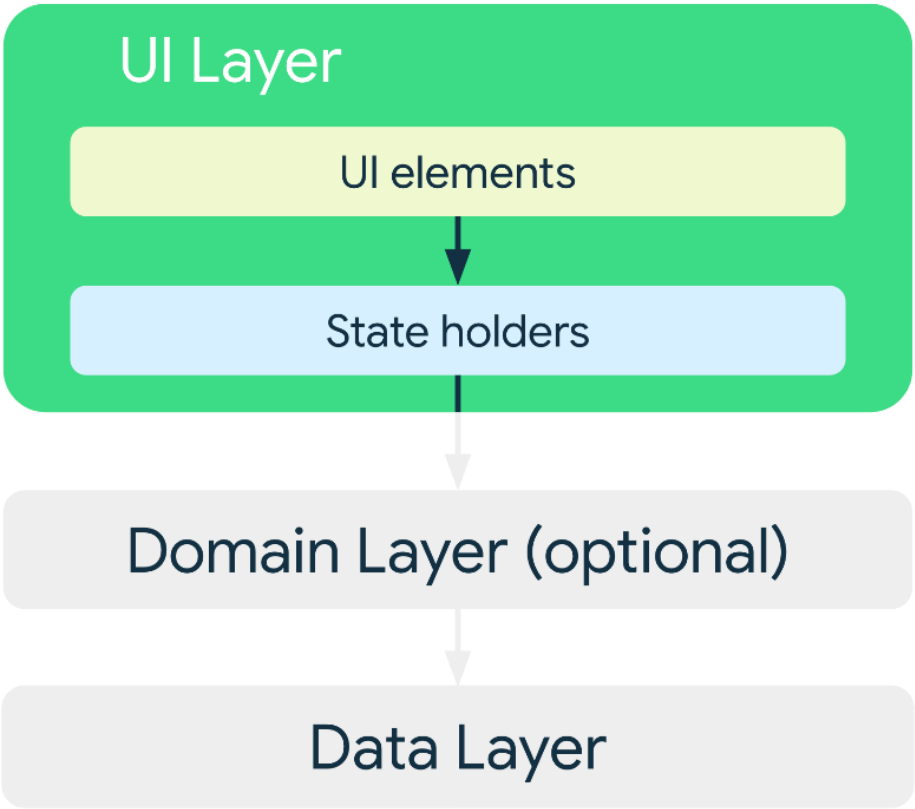
Vai trò của lớp giao diện người dùng (hay lớp trình bày) là hiển thị dữ liệu ứng dụng trên màn hình. Bất cứ khi nào dữ liệu thay đổi, dù là do tương tác của người dùng

(như nhấn nút) hay do đầu vào từ bên ngoài (như phản hồi từ mạng), UI nên cập nhật để phản ánh các thay đổi này.

Lớp UI bao gồm hai thành phần:

Các phần tử UI hiển thị dữ liệu lên màn hình. Xây dựng các phần tử này bằng cách sử dụng các thành phần Views hoặc các hàm Jetpack Compose.

Các thành phần quản lý trạng thái (chẳng hạn như các lớp ViewModel) lưu trữ dữ liệu, cung cấp dữ liệu đó cho UI, và xử lý logic.



Hình 7.2: Các lớp trong kiến trúc ứng dụng hiện đại

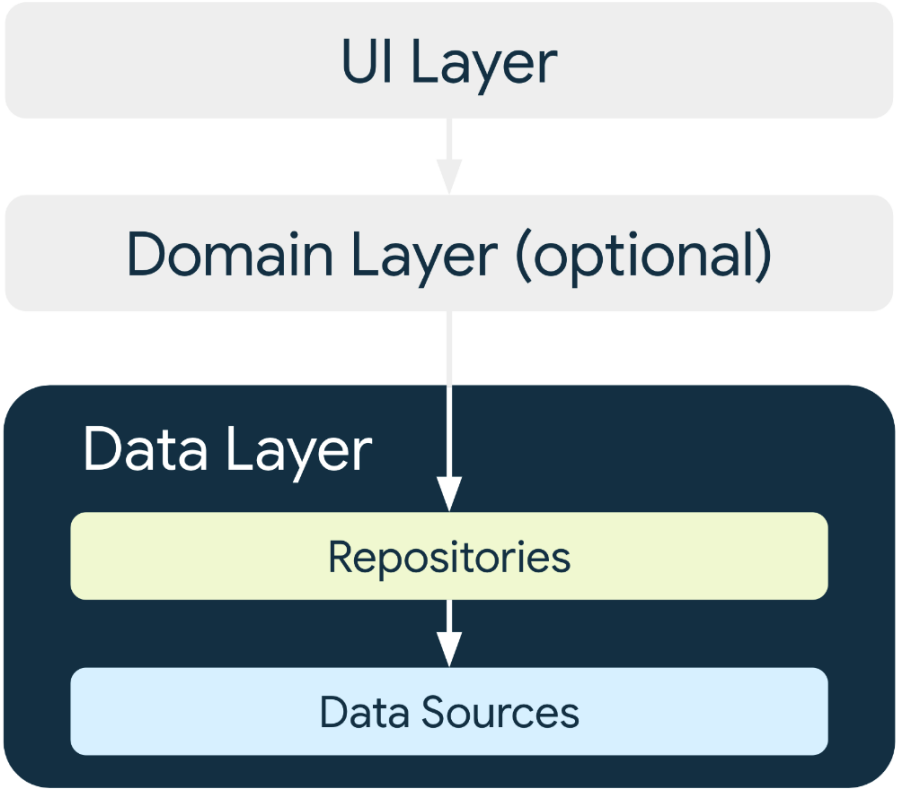
Lớp dữ liệu

Lớp dữ liệu của một ứng dụng là nơi lưu trữ và xử lý logic nghiệp vụ, giúp ứng dụng hoạt động theo đúng mục đích của nó. Logic nghiệp vụ bao gồm các quy tắc và phương thức xác định cách ứng dụng tạo ra, truy xuất, lưu trữ, cập nhật và xóa dữ liệu. Đây là phần quan trọng trong kiến trúc ứng dụng, đảm bảo dữ liệu luôn chính xác và đáng tin cậy khi được hiển thị hoặc sử dụng.

Lớp dữ liệu thường được thiết kế theo mô hình kho lưu trữ (repository pattern), trong đó mỗi kho lưu trữ chịu trách nhiệm quản lý một loại dữ liệu nhất định. Một repository có thể chứa từ không đến nhiều nguồn dữ liệu, chẳng hạn như cơ sở dữ liệu cục bộ (SQLite, Room), API từ xa (Retrofit), SharedPreferences hoặc các tệp nội bộ.

Khi thiết kế lớp dữ liệu, nên tách riêng kho lưu trữ cho từng loại dữ liệu để giúp mã nguồn rõ ràng và dễ bảo trì hơn. Ví dụ, nếu ứng dụng cần xử lý dữ liệu phim, có thể tạo một MoviesRepository chuyên quản lý dữ liệu liên quan đến phim. Nếu ứng dụng có tính năng thanh toán, nên tạo một PaymentsRepository để xử lý các giao dịch và thông tin thanh toán.

Việc phân tách các kho lưu trữ theo từng loại dữ liệu giúp có thể mở rộng ứng dụng một cách dễ dàng mà không làm ảnh hưởng đến các phần khác. Nó cũng giúp việc kiểm thử trở nên hiệu quả hơn, vì có thể kiểm tra từng kho lưu trữ một cách độc lập mà không phụ thuộc vào toàn bộ hệ thống.



Hình 7.3.: Lớp Data

Các lớp Repository chịu trách nhiệm cho các nhiệm vụ sau:

Cung cấp dữ liệu cho các phần khác của ứng dụng.

Tập trung hóa các thay đổi đối với dữ liệu.

Giải quyết xung đột giữa nhiều nguồn dữ liệu khác nhau.

Trừu tượng hóa các nguồn dữ liệu khỏi các phần còn lại của ứng dụng.

Chứa logic nghiệp vụ.

Mỗi lớp nguồn dữ liệu (data source class) nên chỉ có trách nhiệm làm việc với một nguồn dữ liệu duy nhất, có thể là một tệp tin, một nguồn mạng, hoặc một cơ sở dữ liệu

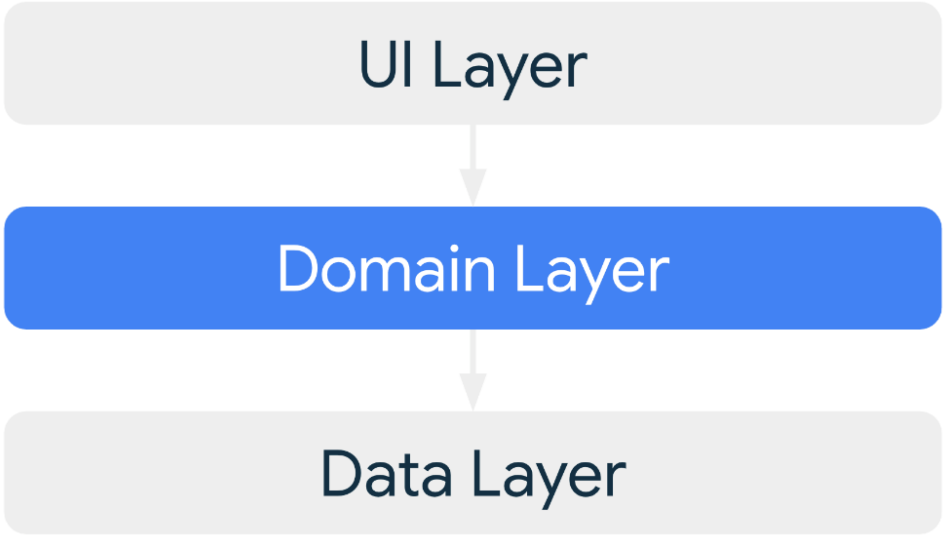
cục bộ. Các lớp nguồn dữ liệu là cầu nối giữa ứng dụng và hệ thống để thực hiện các thao tác dữ liệu.

Để tìm hiểu thêm về lớp này, hãy xem trang lớp dữ liệu.

Lớp miền (Domain layer)

Lớp miền là một lớp tùy chọn nằm giữa lớp giao diện người dùng (UI) và lớp dữ liệu.

Lớp miền chịu trách nhiệm đóng gói các logic nghiệp vụ phức tạp, hoặc các logic nghiệp vụ đơn giản được nhiều ViewModel sử dụng lại. Lớp này là tùy chọn vì không phải ứng dụng nào cũng cần đến nó. Chỉ nên sử dụng lớp miền khi cần thiết—ví dụ, để xử lý sự phức tạp hoặc ưu tiên cho tính tái sử dụng.



Hình 7.4: Lớp Domain

Các lớp trong lớp miền này thường được gọi là use case hoặc interactor. Mỗi use case nên chịu trách nhiệm về một chức năng cụ thể. Ví dụ, ứng dụng có thể có một lớp GetTimeZoneUseCase nếu nhiều ViewModel phụ thuộc vào múi giờ để hiển thị thông điệp phù hợp trên màn hình.

Quản lý các phụ thuộc giữa các thành phần

Các lớp trong ứng dụng phụ thuộc vào các lớp khác để hoạt động đúng cách. có thể sử dụng một trong các mẫu thiết kế sau để thu thập các phụ thuộc của một lớp cụ thể:

Dependency injection (DI): Dependency injection cho phép các lớp định nghĩa các phụ thuộc của chúng mà không cần tự xây dựng các phụ thuộc đó. Vào thời điểm chạy, một lớp khác sẽ chịu trách nhiệm cung cấp các phụ thuộc này.

Service locator: Mẫu thiết kế service locator cung cấp một registry nơi các lớp có thể lấy được các phụ thuộc của mình thay vì tự xây dựng chúng.

Các mẫu thiết kế này cho phép mở rộng mã của mình vì chúng cung cấp các mẫu rõ ràng để quản lý các phụ thuộc mà không phải lặp lại mã hoặc thêm sự phức tạp. Hơn nữa, các mẫu thiết kế này cho phép dễ dàng chuyển đổi giữa các phiên bản kiểm thử và phiên bản sản phẩm.

Chúng tôi khuyến khích sử dụng các mẫu thiết kế dependency injection và thư viện Hilt trong các ứng dụng Android. Hilt tự động xây dựng các đối tượng bằng cách đi qua cây phụ thuộc, cung cấp các đảm bảo về phụ thuộc tại thời gian biên dịch và tạo ra các container phụ thuộc cho các lớp trong Android framework.

7.1.1.7. Các khuyến nghị

Lập trình là một lĩnh vực sáng tạo, và xây dựng ứng dụng Android cũng không ngoại lệ. Có rất nhiều cách để giải quyết một vấn đề; có thể truyền dữ liệu giữa nhiều activity hoặc fragment, lấy dữ liệu từ xa và lưu trữ nó cục bộ cho chế độ ngoại tuyến, hoặc xử lý vô số các tình huống phổ biến khác mà các ứng dụng phức tạp gặp phải.

Mặc dù các khuyến nghị sau đây không bắt buộc, nhưng trong hầu hết các trường hợp, tuân theo chúng sẽ làm cho mã nguồn mạnh mẽ hơn, dễ kiểm thử hơn, và dễ bảo trì hơn trong thời gian dài:

Không lưu trữ dữ liệu trong các thành phần của ứng dụng.

Tránh việc coi các điểm đầu vào của ứng dụng—chẳng hạn như các activity, service và broadcast receiver—là các nguồn dữ liệu. Thay vào đó, chúng chỉ nên phối hợp với các thành phần khác để truy xuất tập hợp con của dữ liệu liên quan đến điểm đầu vào đó. Mỗi thành phần của ứng dụng có vòng đời tương đối ngắn, tùy thuộc vào tương tác của người dùng với thiết bị và tình trạng sức khỏe tổng thể của hệ thống.

Giảm thiểu sự phụ thuộc vào các lớp trong Android.

Các thành phần của ứng dụng nên là những lớp duy nhất phụ thuộc vào các API trong Android framework như Context hoặc Toast. Việc trừu tượng hóa các lớp khác trong ứng dụng khỏi chúng giúp kiểm thử dễ dàng hơn và giảm sự phụ thuộc trong ứng dụng .

Tạo ranh giới trách nhiệm rõ ràng giữa các module khác nhau trong ứng dụng .

Ví dụ, đừng phân tán mã để tải dữ liệu từ mạng qua nhiều lớp hoặc gói trong mã nguồn . Tương tự, đừng định nghĩa nhiều trách nhiệm không liên quan—chẳng hạn như lưu trữ tạm thời dữ liệu và liên kết dữ liệu—trong cùng một lớp. Việc tuân theo kiến trúc ứng dụng được khuyến nghị sẽ giúp trong vấn đề này.

Chỉ công khai những gì cần thiết nhất từ mỗi module.

Ví dụ, đừng tạo ra một lối tắt để công khai chi tiết triển khai nội bộ từ một module. có thể tiết kiệm một chút thời gian ngắn hạn, nhưng sau đó có thể sẽ phải gánh chịu nợ kỹ thuật nhiều lần khi mã nguồn phát triển.

Tập trung vào phần cốt lõi độc đáo của ứng dụng để nó nổi bật so với các ứng dụng khác.

Đừng sáng tạo lại bánh xe bằng cách viết lại những đoạn mã mẫu giống nhau nhiều lần. Thay vào đó, hãy tập trung thời gian và năng lượng vào những gì làm cho ứng dụng trở nên độc đáo, và để các thư viện Jetpack và các thư viện được khuyến nghị khác xử lý những đoạn mã mẫu lặp lại.

Cân nhắc cách làm cho mỗi phần của ứng dụng có thể kiểm thử độc lập.

Ví dụ, có một API được định nghĩa rõ ràng để lấy dữ liệu từ mạng giúp việc kiểm thử module lưu trữ dữ liệu đó trong cơ sở dữ liệu cục bộ dễ dàng hơn. Nếu thay vào đó, trộn lẫn logic từ hai module này vào cùng một chỗ, hoặc phân tán mã mạng trên toàn bộ mã nguồn, việc kiểm thử sẽ trở nên khó khăn hơn—nếu không muốn nói là không thể—để kiểm thử hiệu quả.

Các kiểu chịu trách nhiệm cho chính sách đồng thời của mình.

Nếu một kiểu đang thực hiện công việc chặn lâu dài, nó nên chịu trách nhiệm chuyển công việc tính toán đó sang đúng luồng. Kiểu cụ thể đó biết loại tính toán mà nó đang thực hiện và trong luồng nào nó nên được thực hiện. Các kiểu nên an toàn với luồng chính, nghĩa là chúng an toàn khi được gọi từ luồng chính mà không làm chặn nó.

Lưu trữ càng nhiều dữ liệu liên quan và mới nhất càng tốt.

Bằng cách đó, người dùng có thể tận hưởng chức năng của ứng dụng ngay cả khi thiết bị của họ ở chế độ ngoại tuyến. Hãy nhớ rằng không phải tất cả người dùng đều có kết nối liên tục và tốc độ cao—và ngay cả khi họ có, họ cũng có thể bị mất tín hiệu ở những nơi đông người.

7.1.1.8. Lợi ích của Kiến trúc

Việc triển khai một Kiến trúc tốt trong ứng dụng mang lại nhiều lợi ích cho dự án và các nhóm kỹ thuật:

Nó cải thiện khả năng bảo trì, chất lượng và độ ổn định của toàn bộ ứng dụng.

Nó cho phép ứng dụng mở rộng. Nhiều người và nhiều nhóm có thể đóng góp cho cùng một mã nguồn với xung đột mã tối thiểu.

Nó giúp đẩy nhanh quá trình gia nhập. Vì kiến trúc mang lại sự nhất quán cho dự án , các thành viên mới của nhóm có thể nhanh chóng nắm bắt và làm việc hiệu quả hơn trong thời gian ngắn hơn.

Nó dễ kiểm thử hơn. Một Kiến trúc tốt khuyến khích các kiểu đơn giản hơn, thường dễ kiểm thử hơn.

Các lỗi có thể được điều tra một cách có phương pháp với các quy trình được định nghĩa rõ ràng.

Đầu tư vào Kiến trúc cũng có tác động trực tiếp đến người dùng . Họ được hưởng lợi từ một ứng dụng ổn định hơn và nhiều tính năng hơn nhờ vào một nhóm kỹ thuật năng suất hơn. Tuy nhiên, Kiến trúc cũng đòi hỏi một khoản đầu tư thời gian ban đầu. Để giúp chứng minh khoản đầu tư này với các thành viên khác trong công ty, hãy xem các nghiên cứu trường hợp nơi các công ty khác chia sẻ câu chuyện thành công của họ khi có một kiến trúc tốt trong ứng dụng của họ.

7.2. Kiến trúc MVVM với ViewModel và LiveData

7.2.1. Kiến trúc MVVM

7.2.1.1. Giới thiệu về MVVM

Mô hình MVVM (Model-View-ViewModel) là một kiến trúc thiết kế phần mềm phổ biến trong phát triển ứng dụng Android.

MVVM tách biệt rõ ràng giữa giao diện người dùng (UI) và logic nghiệp vụ, giúp tăng khả năng bảo trì, kiểm thử và tái sử dụng mã nguồn.

7.2.1.2. Các thành phần chính trong MVVM

Model

- Là lớp chịu trách nhiệm lưu trữ và xử lý dữ liệu.
- Bao gồm logic nghiệp vụ và tương tác với cơ sở dữ liệu hoặc API.
- Vai trò: Cung cấp dữ liệu và phản hồi khi dữ liệu thay đổi.

View

- Đại diện cho giao diện người dùng (UI).
- Nhận dữ liệu từ ViewModel để hiển thị và gửi sự kiện từ người dùng về cho ViewModel.
- Ví dụ: Activity, Fragment, hoặc giao diện trong Jetpack Compose.

ViewModel

- Lớp trung gian giữa Model và View.
- Chịu trách nhiệm xử lý logic giao diện và cung cấp dữ liệu cho View.
- Sử dụng LiveData hoặc StateFlow để truyền dữ liệu từ Model đến View.

7.2.1.3. Luồng hoạt động trong MVVM

Từ View đến ViewModel:

- View gửi sự kiện từ người dùng (như nhấn nút) đến ViewModel.
- ViewModel xử lý sự kiện và tương tác với Model.

Từ Model đến ViewModel:

- Model thay đổi dữ liệu (ví dụ: API trả về kết quả).
- ViewModel lắng nghe thay đổi từ Model và cung cấp dữ liệu cho View.

Từ ViewModel đến View:

- ViewModel gửi dữ liệu cập nhật đến View.
- View hiển thị thông tin mới.

7.2.1.4. Lợi ích của MVVM

Tăng khả năng bảo trì: Tách biệt logic nghiệp vụ và giao diện, giúp dễ dàng thay đổi UI mà không ảnh hưởng đến logic.

Kiểm thử dễ dàng: ViewModel không phụ thuộc trực tiếp vào View, cho phép kiểm thử logic giao diện mà không cần UI thực.

Tái sử dụng mã nguồn: ViewModel có thể được tái sử dụng bởi nhiều View.

Quản lý trạng thái hiệu quả: ViewModel lưu giữ trạng thái UI ngay cả khi View bị hủy và tái tạo.

7.2.1.5. Cách triển khai MVVM trong Android

Thêm thư viện cần thiết:

```
implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.0"
implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.6.0"
```

Tạo Model:

```
data class Student(val id: Int, val name: String)
```

Tạo ViewModel:

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class StudentViewModel : ViewModel() {
  private val _students = MutableLiveData<List<Student>>()
  val students: LiveData<List<Student>> get() = _students

  fun loadStudents() {
```

```
    _students.value = listOf(
      Student(1, "Nam"),
      Student(2, "Hà"),
      Student(3, "Mai")
    )
  }
}
```

Tạo View (Activity):

```
import androidx.activity.viewModels

class MainActivity : AppCompatActivity() {
  private val studentViewModel: StudentViewModel by viewModels()

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    studentViewModel.students.observe(this) { students ->
      textView.text = students.joinToString("\n") { it.name }
    }
    studentViewModel.loadStudents()
  }
}
```

Giao diện XML:

```
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">

  <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
    android:text="Danh sách sinh viên" />
</LinearLayout>
```

7.2.1.6. Lưu ý khi sử dụng MVVM

- Quản lý vòng đời: ViewModel gắn liền với vòng đời của Activity hoặc Fragment.
- Tránh logic trong View: Chuyển mọi logic nghiệp vụ vào ViewModel.
- Sử dụng LiveData/StateFlow: Đảm bảo giao diện tự động cập nhật khi dữ liệu thay đổi.

7.2.1.7. Kết hợp MVVM với các công cụ hiện đại

- Hilt/Dagger: Quản lý Dependency Injection.
- Room: Kết hợp với Model để lưu trữ dữ liệu cục bộ.
- Navigation Component: Điều hướng giữa các View trong ứng dụng.
- Jetpack Compose: Tích hợp MVVM với giao diện hiện đại bằng cách sử dụng State.

7.2.2. ViewModel

7.2.2.1. Giới thiệu về ViewModel

ViewModel là một thành phần trong Android Architecture Components, được thiết kế để lưu trữ và quản lý dữ liệu liên quan đến UI. Điều này giúp duy trì dữ liệu khi có các thay đổi cấu hình như xoay màn hình.

- ViewModel giúp tách biệt giữa logic xử lý dữ liệu và logic giao diện người dùng.
- Nó cũng giúp chia sẻ dữ liệu giữa các thành phần như Activity hoặc Fragment.
- ViewModel có vòng đời dài hơn Activity hoặc Fragment, do đó dữ liệu sẽ không bị mất khi các sự kiện như xoay màn hình xảy ra.

Lợi ích của ViewModel

- **Duy trì trạng thái**: Giúp giữ lại dữ liệu khi có sự thay đổi cấu hình mà không cần khởi tạo lại.
- **Tách biệt logic**: Logic kinh doanh và logic giao diện được tách riêng, giúp cho code dễ bảo trì và mở rộng.

7.2.2.2. Cách tạo ViewModel

Đầu tiên, cần thêm thư viện ViewModel vào dự án:

```
dependencies {
  def lifecycle_version = "2.5.0"
```

```
  implementation "androidx.lifecycle:lifecycle-viewmodel-
ktx:$lifecycle_version"
}
```

Sau đó, tạo một lớp ViewModel bằng cách kế thừa ViewModel. Ví dụ đơn giản:

```
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
  var count = 0

  fun incrementCount() {
    count++
  }
}
```

Trong Activity hoặc Fragment, sử dụng ViewModelProvider để truy cập ViewModel:

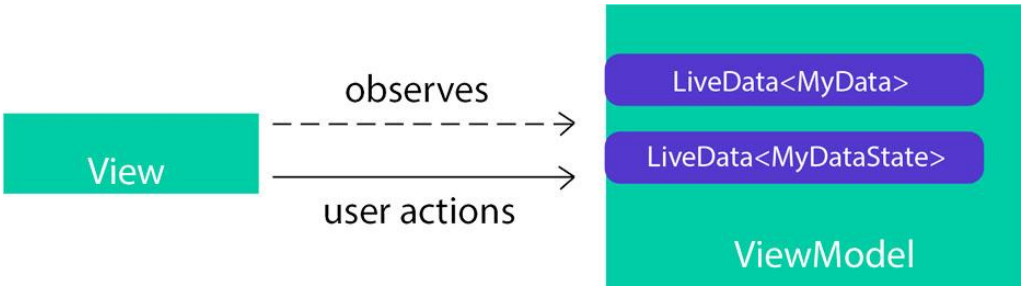
```
import androidx.activity.viewModels
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
  private val viewModel: CounterViewModel by viewModels()

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    viewModel.incrementCount()
    println("Current count: ${viewModel.count}")
  }
}
```

7.2.2.3. Sử dụng LiveData trong ViewModel



Hình 7.5: Quy trình hoạt động với LiveData

LiveData giúp quản lý dữ liệu theo cách phản ứng lại sự thay đổi và kết hợp với ViewModel giúp quản lý dữ liệu hiệu quả hơn.

Trong ViewModel, sử dụng LiveData để lưu trữ và quản lý dữ liệu có thể quan sát được:

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
  private val _count = MutableLiveData(0)
  val count: LiveData<Int> = _count

  fun incrementCount() {
    _count.value = (_count.value ?: 0) + 1
  }
}
```

Trong Activity hoặc Fragment, quan sát LiveData để cập nhật giao diện người dùng:

```
import android.os.Bundle
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.Observer

class MainActivity : AppCompatActivity() {
  private val viewModel: CounterViewModel by viewModels()

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

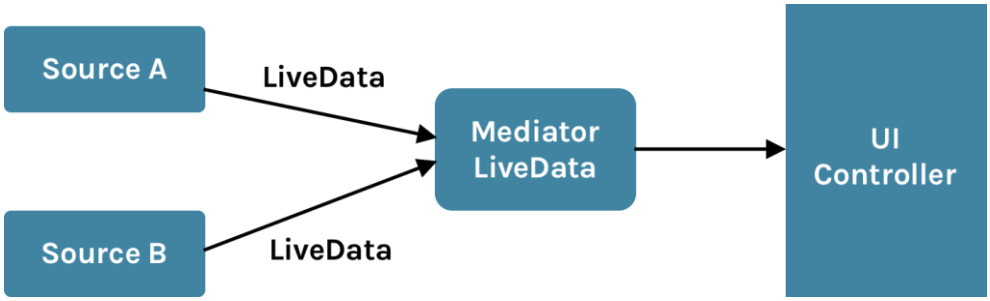
```
    setContentView(R.layout.activity_main)

    viewModel.count.observe(this, Observer { count ->
      println("Current count: $count")
    })

    viewModel.incrementCount()
  }
}
```

Sử dụng ViewModel giúp quản lý dữ liệu một cách hiệu quả và duy trì dữ liệu khi thay đổi cấu hình. Kết hợp với LiveData, ViewModel giúp phản ứng linh hoạt với các thay đổi dữ liệu và giúp giữ cho logic kinh doanh tách biệt với logic giao diện, tạo nên ứng dụng có cấu trúc rõ ràng hơn.

7.2.3. LiveData



Hình 7.6: LiveData

7.2.3.1. Giới thiệu về LiveData

LiveData là một thành phần trong Android Jetpack giúp quản lý dữ liệu theo cách nhận thức vòng đời (Lifecycle-aware).

LiveData tự động cập nhật giao diện người dùng (UI) khi dữ liệu thay đổi, giảm thiểu sự phức tạp trong việc đồng bộ hóa dữ liệu với giao diện.

7.2.3.2. Đặc điểm chính của LiveData

Nhận thức vòng đời: LiveData chỉ cập nhật giao diện khi LifecycleOwner (ví dụ: Activity, Fragment) ở trạng thái STARTED hoặc RESUMED.

Tránh rò rỉ bộ nhớ và đảm bảo tính an toàn.

Dễ dàng quan sát dữ liệu: Giao diện người dùng có thể đăng ký làm observer để nhận thông báo khi dữ liệu thay đổi.

Tích hợp tự động: LiveData tích hợp dễ dàng với các thành phần như ViewModel, Room, giúp đồng bộ dữ liệu giữa tầng logic và giao diện.

7.2.3.3. Luồng hoạt động của LiveData

- Model hoặc các nguồn dữ liệu khác cập nhật giá trị của LiveData.
- LiveData thông báo các thay đổi đến tất cả các observer đã đăng ký.
- View nhận dữ liệu mới từ LiveData và tự động cập nhật giao diện người dùng.

7.2.3.4. Cách triển khai LiveData

Trong ViewModel:

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
class UserViewModel : ViewModel() {
  private val _userName = MutableLiveData<String>()
  val userName: LiveData<String> get() = _userName

  fun updateUserName(newName: String) {
    _userName.value = newName
  }
}
```

Trong View (Activity):

```
  import androidx.activity.viewModels
  class MainActivity : AppCompatActivity() {
    private val userViewModel: UserViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      setContentView(R.layout.activity_main)
      userViewModel.userName.observe(this) { name ->
        textView.text = "Xin chào, $name!"
      }
      button.setOnClickListener {
        userViewModel.updateUserName("Nam")
      }
    }
  }
```

Giao diện XML:

```
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">

  <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Xin chào!" />

  <Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Thay đổi tên" />
</LinearLayout>
```

7.2.3.5. Lợi ích của LiveData

Tự động đồng bộ giao diện: LiveData tự động gửi dữ liệu mới đến giao diện khi dữ liệu thay đổi.

Quản lý vòng đời hiệu quả: LiveData chỉ hoạt động khi View ở trạng thái hợp lệ, giúp tránh rò rỉ bộ nhớ.

Tăng tính bảo trì: Tách biệt rõ ràng giữa dữ liệu và giao diện.

Dễ dàng kết hợp với Jetpack: LiveData hoạt động tốt với ViewModel, Room, và Navigation.

7.2.4. Repository

7.2.4.1. Giới thiệu về Repository

Repository là một lớp trung gian trong kiến trúc MVVM, chịu trách nhiệm quản lý truy xuất dữ liệu từ các nguồn khác nhau như:

- Cơ sở dữ liệu cục bộ (Room).
- API từ máy chủ.
- Bộ nhớ đệm.

Repository giúp tách biệt logic truy xuất dữ liệu khỏi ViewModel, đảm bảo mã nguồn dễ bảo trì và mở rộng.

7.2.4.2. Vai trò của Repository

Cung cấp dữ liệu hợp nhất: Repository thu thập dữ liệu từ nhiều nguồn và cung cấp dữ liệu đã xử lý cho ViewModel.

Tách biệt logic dữ liệu và giao diện: ViewModel không trực tiếp tương tác với nguồn dữ liệu, mà thông qua Repository.

Xử lý logic liên quan đến dữ liệu: Kết hợp, lọc, hoặc xử lý dữ liệu trước khi gửi đến ViewModel.

7.2.4.3. Cách triển khai Repository

Tạo lớp Repository:

```
class UserRepository {
  fun getUsers(): List<String> {
    return listOf("Nam", "Hà", "Mai")
  }
}
```

Sử dụng Repository trong ViewModel:

```
class UserViewModel(private val userRepository: UserRepository) :
ViewModel() {
  private val _users = MutableLiveData<List<String>>()
  val users: LiveData<List<String>> get() = _users

  fun loadUsers() {
    _users.value = userRepository.getUsers()
  }
}
```

Kết hợp Dependency Injection (DI):

Sử dụng Hilt để cung cấp Repository cho ViewModel.

```
@HiltViewModel
class UserViewModel @Inject constructor(
  private val userRepository: UserRepository
) : ViewModel() {
  fun loadUsers() {
    // Logic sử dụng repository
```

```
  }
}
```

Tích hợp với View (Activity):

```
val userViewModel: UserViewModel by viewModels()

userViewModel.users.observe(this) { users ->
  textView.text = users.joinToString("\n")
}

userViewModel.loadUsers()
```

7.2.4.4. Lợi ích của Repository

Đơn giản hóa ViewModel: ViewModel chỉ tập trung vào logic giao diện và không xử lý logic truy xuất dữ liệu.

Tăng tính tái sử dụng: Repository có thể được sử dụng bởi nhiều ViewModel khác nhau.

Tăng khả năng kiểm thử: Có thể kiểm thử logic trong Repository một cách độc lập.

Quản lý dữ liệu hiệu quả: Kết hợp dữ liệu từ nhiều nguồn như API và cơ sở dữ liệu cục bộ.

Bài tập bổ trợ

Bài tập thực hành cơ bản

1. Tạo ViewModel quản lý danh sách chuỗi và hiển thị danh sách trên giao diện.
2. Sử dụng LiveData để quan sát thay đổi và cập nhật UI.
3. Tích hợp Repository để quản lý dữ liệu từ cơ sở dữ liệu và mạng.
4. Sử dụng Dependency Injection với Hilt để quản lý các thành phần trong ứng dụng.
5. Tạo ứng dụng sử dụng mô hình MVVM cơ bản.

Bài mẫu 1: Tạo ViewModel quản lý danh sách

Mục tiêu: Tạo ứng dụng hiển thị danh sách sinh viên từ ViewModel và cập nhật giao diện bằng LiveData.

Bước thực hiện:

Thêm Dependency ViewModel:

```
implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.0"
implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.6.0"
```

Tạo ViewModel:

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class StudentViewModel : ViewModel() {
  private val _students = MutableLiveData<List<String>>()
  val students: LiveData<List<String>> get() = _students

  init {
    loadStudents()
  }

  private fun loadStudents() {
    _students.value = listOf("Nam", "Hà", "Mai", "Quang")
  }
}
```

Hiển thị danh sách trên giao diện:

```
// Trong Activity hoặc Fragment
val viewModel: StudentViewModel by viewModels()

viewModel.students.observe(this) { studentList ->
  textView.text = studentList.joinToString("\n")
}
```

Bài mẫu 2: Ứng dụng Đếm số sử dụng LiveData & ViewModel

Mục tiêu:

Tạo một **ViewModel** chứa giá trị **LiveData** để đếm số lần người dùng bấm nút.

Cập nhật **UI** tự động khi dữ liệu thay đổi.

Tạo một lớp CounterViewModel.kt để lưu trữ và quản lý dữ liệu.

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
  private val _count = MutableLiveData(0) // LiveData chứa giá trị đếm
```

```
  val count: LiveData<Int> = _count // Expose chỉ để đọc từ ngoài

  fun increaseCount() {
    _count.value = (_count.value ?: 0) + 1
  }
}
```

Giao diện XML (activity_main.xml**)**

Tạo TextView hiển thị số lần bấm và Button để tăng số.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:gravity="center"
  android:orientation="vertical">

  <TextView
    android:id="@+id/tvCounter"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="0"
    android:textSize="30sp"/>

  <Button
    android:id="@+id/btnIncrease"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tăng số"/>
</LinearLayout>
```

MainActivity.kt

```
import android.os.Bundle
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.Observer
import android.widget.Button
```

```
import android.widget.TextView

class MainActivity : AppCompatActivity() {

  private val counterViewModel: CounterViewModel by viewModels()

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val tvCounter = findViewById<TextView>(R.id.tvCounter)
    val btnIncrease = findViewById<Button>(R.id.btnIncrease)

    // Quan sát LiveData, UI sẽ cập nhật tự động khi giá trị thay
                đổi
    counterViewModel.count.observe(this, Observer { newCount ->
      tvCounter.text = newCount.toString()
    })

    // Khi bấm nút, gọi ViewModel để tăng số
    btnIncrease.setOnClickListener {
      counterViewModel.increaseCount()
    }
  }}
```